📦 L04: Nested Constructions

Boxes Inside Boxes

Neon City Cyber Academy

Module 1: Python Basics

👥 Your Team

Emily Chen - *Handler*

> "Today we combine operations. Think of it like Russian nesting dolls - one function inside another."

Cole Martinez - *Tech Lead*

> "`int(input())` is the most powerful pattern you'll learn this week. Master this, master data flow."

🔄 Review: input()

Get text from the user:

```
name = input("What's your name? ")
print(f"Hello, {name}!")
```

Problem: `input()` ALWAYS returns a **string**, even for numbers!

```
age = input("Age: ")
# If user types "14", age = "14" (text!)
```

🔢 Review: int() and Type Conversion

Convert strings to numbers:

```
text_num = "42"
real_num = int(text_num)

print(text_num + text_num)    # 4242 (string concat)
print(real_num + real_num)    # 84 (math)
```

Type conversion functions:

- `int()` → integer (whole number)
- `float()` → decimal number
- `str()` → string (text)

💡 The Problem

```
# Get user's age
age_text = input("Enter age: ")

# Convert to number
age_number = int(age_text)

# Use in calculation
next_year = age_number + 1
print(f"Next year you'll be {next_year}")
```

That's 3 lines for one simple task!

Can we do better? YES! 🎯

🎁 Nested Functions: The Solution

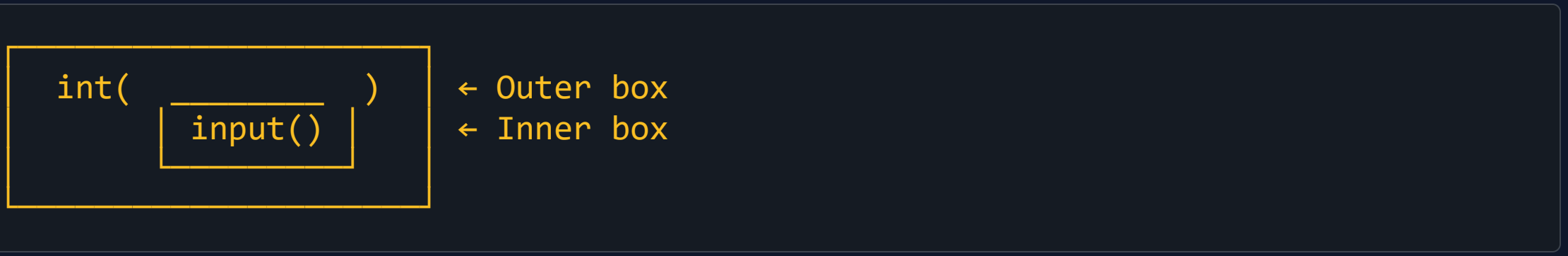
Combine `int()` and `input()` in ONE line:

```
age = int(input("Enter age: "))
next_year = age + 1
print(f"Next year you'll be {next_year}")
```

One variable, one line!

💬 Emily's Nesting Explanation:

> "Think of functions as boxes:"



> "Python executes from INSIDE OUT:"
>
> 1. `input()` runs first → gets text
> 2. `int()` runs second → converts to number
> 3. Result stored in variable

🔄 Execution Order

```
score = int(input("Score: "))
```

Step-by-step:

1. User types: `95`
2. `input()` returns: `"95"` (string)
3. `int("95")` converts to: `95` (number)
4. `score` gets: `95`

Reading code: Inside → Outside

Execution: Inside → Outside

💬 Cole's Rule:

> "The INNERMOST function always executes FIRST."

```
result = int(input("Number: "))

Order of execution:
  1st: input("Number: ")      ← Innermost
  2nd: int( ... )             ← Outer
  3rd: result = ...           ← Assignment
```

> "Like peeling an onion - start from the center!"

🧮 Nested Math Operations

```
x = 5
y = 3

# Multiple operations in one line
result = ((x + y) * 2) - 1

Step by step:
  (5 + 3) = 8
  8 * 2 = 16
  16 - 1 = 15
```

Parentheses control order - innermost first!

🎯 Practice: User Calculator

```
num1 = int(input("First number: "))
num2 = int(input("Second number: "))

sum_result = num1 + num2
product = num1 * num2

print(f"{num1} + {num2} = {sum_result}")
print(f"{num1} × {num2} = {product}")
```

Two nested constructions: `int(input())` used twice!

🔢 Float Input

For decimal numbers:

```
price = float(input("Price: $"))
quantity = int(input("Quantity: "))

total = price * quantity
print(f"Total: ${total}")
```

User enters: `19.99` and `3`

Output: `Total: $59.97`

🎨 Complex Nesting

You can nest MULTIPLE levels:

```
# Get number, double it, convert to string
text = str(int(input("Number: ")) * 2)

Execution order:
  1. input("Number: ") → "5"
  2. int("5") → 5
  3. 5 * 2 → 10
  4. str(10) → "10"
  5. text = "10"
```

Rarely needed, but possible!

💬 Emily Warns:

> "Don't nest TOO much - code should be readable!"

```
# ❌ TOO COMPLEX (hard to read)
x = int(str(float(input("???"))))

# ✅ BETTER (clear steps)
user_input = input("Enter number: ")
as_float = float(user_input)
as_int = int(as_float)
```

> "If you need more than 2 levels, use multiple lines."

🐞 Common Errors

Error #1: User enters text instead of number

```
age = int(input("Age: "))
# User types: "hello"
# ValueError: invalid literal for int()
```

Fix: Validate input (we'll learn this with conditionals!)

Error #2: Forgetting Conversion

```
num = input("Number: ")  # Gets "5" as string
doubled = num * 2
print(doubled)           # Output: 55 (not 10!)
```

Remember: `input()` returns strings!

Always convert with `int()` or `float()` for math.

🎯 Challenge: Temperature Converter

```
celsius = float(input("Celsius: "))
fahrenheit = (celsius * 9/5) + 32

print(f"{celsius}°C = {fahrenheit}°F")
```

Test it:

- Input: `0` → Output: `32°F`
- Input: `100` → Output: `212°F`

Project: BMI Calculator

```
print("=== BMI Calculator ===")

weight = float(input("Weight (kg): "))
height = float(input("Height (m): "))

bmi = weight / (height ** 2)

print(f"Your BMI: {bmi:.1f}")
```

Real-world nesting: `float(input())` + math operations!

🔄 String Nesting

Combine string methods:

```
name = input("Name: ").strip().upper()

Execution:
  1. input("Name: ") → "  alex  "
  2. .strip() → "alex"
  3. .upper() → "ALEX"
```

Method chaining = nested operations!

💬 Cole's Advanced Pattern:

```
# Format user input immediately
username = input("Username: ").lower().strip()

# Get and validate number in one line
count = abs(int(input("Count: ")))
# abs() makes it positive

# Multiple conversions
data = str(round(float(input("Price: ")), 2))
# Get decimal, round to 2 places, convert to string
```

🎲 Random Nested Example

```
import random

# Generate random number and convert to string
secret = str(random.randint(1, 100))

# Get guess and convert to int
guess = int(input("Guess (1-100): "))

# Compare
if guess == int(secret):
    print("Correct!")
```

Nested randomness + user input!

📊 Multiple Inputs in One Line

```
# Get two numbers at once
x, y = int(input("X: ")), int(input("Y: "))

# This is actually:
# x = int(input("X: "))
# y = int(input("Y: "))
```

Advanced shortcut - use sparingly!

✅ Lesson Complete!

You Mastered:

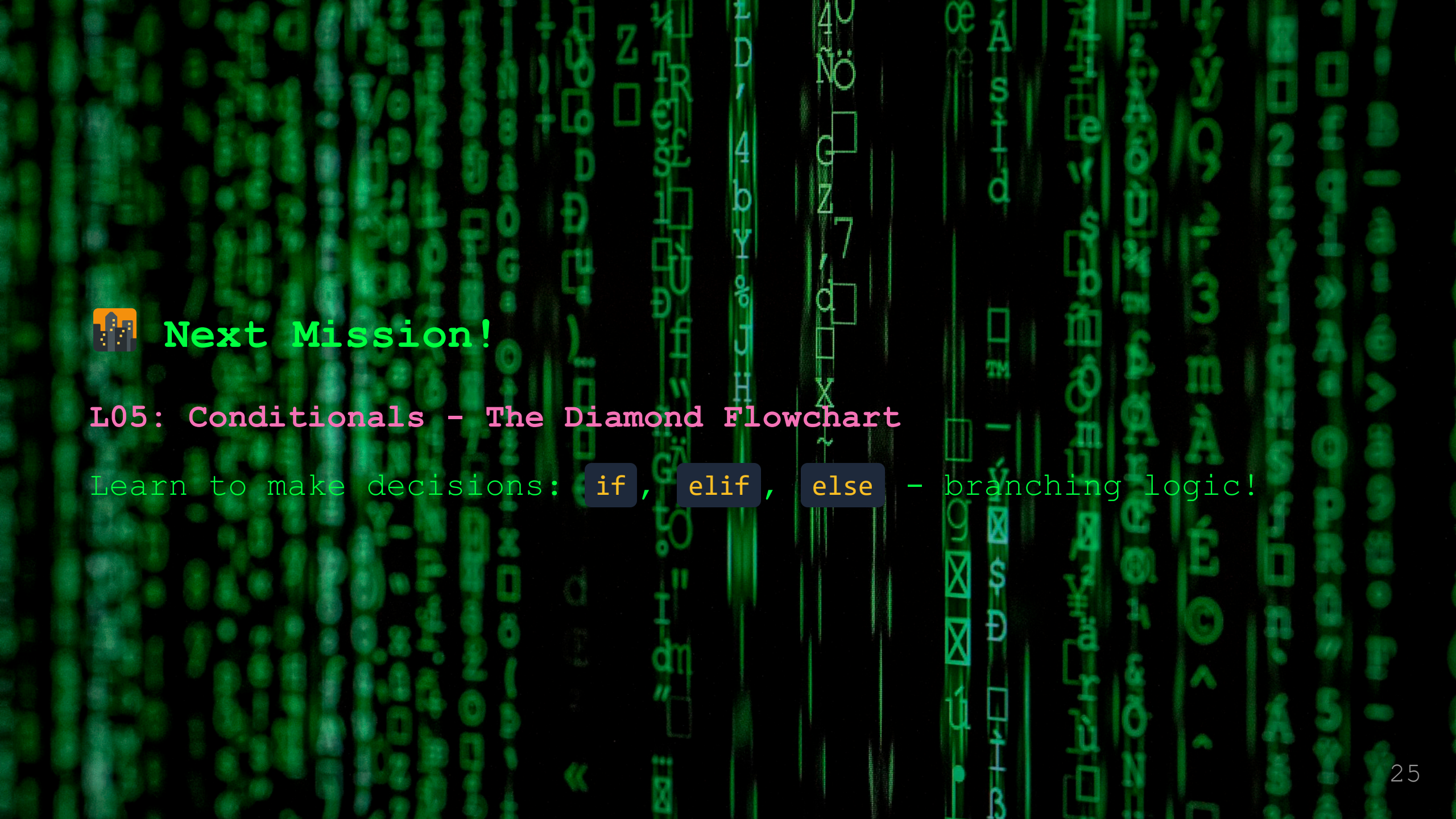
- ✅ `int(input())` pattern (boxes inside boxes)
- ✅ Execution order (inside → outside)
- ✅ `float(input())` for decimals
- ✅ Method chaining (string nesting)
- ✅ Multi-level nesting (when appropriate)
- ✅ Common input errors to avoid

🏆 Your Assignment

Create an **Order Calculator:**

1. Get item name (string, with `.strip()`)
2. Get price (float)
3. Get quantity (int)
4. Calculate total: `price * quantity`
5. Calculate tax: `total * 0.15`
6. Calculate final: `total + tax`
7. Display formatted receipt

Use nested constructions where appropriate!



Next Mission!

L05: Conditionals - The Diamond Flowchart

Learn to make decisions: `if`, `elif`, `else` - branching logic!